CASE STUDY 1: E-COMMERCE PRODUCT SORTING USING DIVIDE AND CONQUER STRATEGY

1. Introduction

E-commerce platforms such as Amazon and Flipkart contain thousands or millions of products. Customers need to sort these products based on price, ratings, popularity, or discounts to find suitable items quickly.

The Divide and Conquer strategy is an algorithm design technique that breaks a large problem into smaller subproblems, solves them independently, and combines their results to obtain the final solution.

In this case study, **Merge Sort** is used to sort e-commerce products by price in ascending order.

2. Problem Statement

An e-commerce website has a list of products with different prices. Sorting these products manually is inefficient and time-consuming.

The objective is to design an efficient algorithm that sorts products from lowest to highest price using the Divide and Conquer strategy.

3. Objectives

- To understand the Divide and Conquer strategy.
- To implement Merge Sort for product sorting.
- To improve the efficiency of product searching and browsing.
- To analyze the time and space complexity of the algorithm.

4. Proposed System

The system accepts a list of products with their prices and sorts them in ascending order.

The Divide and Conquer approach consists of three steps:

1. **Divide:** Split the product list into two halves.
2. **Conquer:** Recursively sort both halves.
3. **Combine:** Merge the sorted halves into one sorted list.

5. Example Dataset

Product	Price (₹)
Headphones	1,500
Keyboard	800
Mouse	500
Monitor	8,000
Webcam	2,000
USB Cable	200

Expected sorted output:

Rank	Product	Price (₹)
1	USB Cable	200
2	Mouse	500
3	Keyboard	800
4	Headphones	1,500

5	Webcam	2,000
6	Monitor	8,000

6. Algorithm

Merge Sort Algorithm

1. Start with the list of products.
2. If the list contains zero or one product, return it.
3. Divide the list into two halves.
4. Apply Merge Sort recursively to each half.
5. Merge the two sorted halves by comparing product prices.
6. Display the final sorted list.
7. Stop.

7. Python Implementation

```
def merge_sort(products):
    if len(products) <= 1:
        return products

    mid = len(products) // 2
    left = merge_sort(products[:mid])
    right = merge_sort(products[mid:])

    return merge(left, right)


def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][1] <= right[j][1]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


products = [
    ("Headphones", 1500),
    ("Keyboard", 800),
    ("Mouse", 500),
    ("Monitor", 8000),
    ("Webcam", 2000),
    ("USB Cable", 200)
]

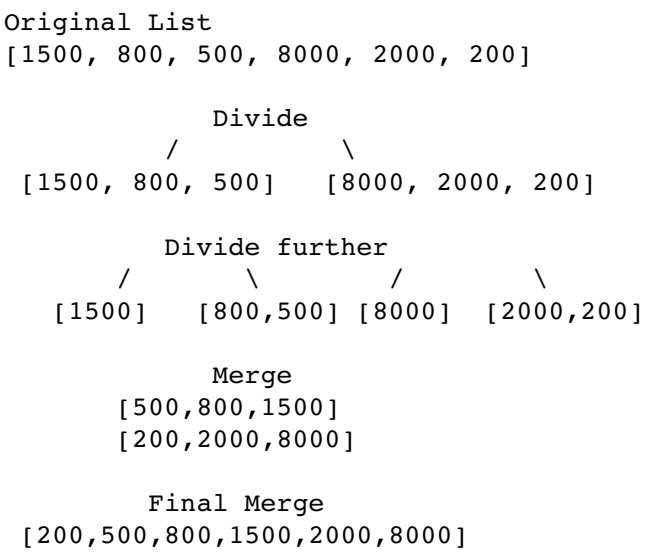
sorted_products = merge_sort(products)

print("Products sorted by price:")
for name, price in sorted_products:
    print(f"{name}: Rs. {price}")
```

8. Working of the Algorithm

The original list is divided into smaller lists until each list contains one product.

For example:



The algorithm compares the prices while merging the smaller sorted lists to produce the final result.

9. Time and Space Complexity

- **Best-case time complexity:** O(n log n)
- **Average-case time complexity:** O(n log n)
- **Worst-case time complexity:** O(n log n)
- **Auxiliary space complexity:** O(n) for the merging process in this implementation.

Here, *n* represents the number of products.

10. Advantages

- Efficient for large product datasets.
- Provides predictable O(n log n) sorting performance.
- Can be adapted to sort by price, ratings, discounts, and popularity.
- Merge Sort is stable, so products with equal prices retain their relative order.

11. Limitations

- Requires additional memory for merging.
- May be less suitable than simpler algorithms for very small datasets.
- Real e-commerce applications may require database indexing, filtering, pagination, and distributed sorting for large catalogues.

12. Applications

- Sorting products by price.
- Ranking products by customer ratings.
- Arranging products by discount percentage.
- Sorting search results by relevance or popularity.
- Organizing inventory data.

13. Conclusion

The Divide and Conquer strategy efficiently solves the e-commerce product-sorting problem by breaking a large list into smaller subproblems. Merge Sort provides O(n log n) time complexity and is suitable for sorting large datasets. This approach improves the organization of product listings and helps customers find relevant products more conveniently.

CASE STUDY 2: GOOGLE CALENDAR APPLICATION USING GREEDY STRATEGY

1. Introduction

Google Calendar is a digital calendar application that allows users to schedule meetings, classes, appointments, reminders, and other activities.

When several events compete for the same time slots, users need to select events that fit within their available schedule. A Greedy algorithm can help solve a simplified version of this problem by selecting compatible events according to a locally optimal rule.

In this case study, the **Activity Selection Problem** is used to demonstrate the Greedy strategy in a calendar-scheduling scenario.

2. Problem Statement

A user has multiple events scheduled throughout the day, but some events overlap. The user wants to attend the maximum possible number of non-overlapping events.

The objective is to select the largest set of compatible events using a Greedy algorithm.

3. Objectives

- To understand the Greedy algorithm design technique.
- To apply the Activity Selection Problem to calendar scheduling.
- To select the maximum number of non-overlapping events.
- To analyze the time and space complexity of the algorithm.

4. Proposed System

The system receives a list of events, each having a start time and an end time.

The Greedy strategy used is:

Select the event that finishes earliest among the remaining compatible events.

Choosing the earliest-finishing event leaves as much time as possible for subsequent events.

The algorithm assumes that all events have equal importance and that the objective is to maximize the number of events attended.

5. Example Dataset

Event	Start Time	End Time
A – Team Meeting	9:00 AM	10:00 AM
B – Project Review	9:30 AM	11:00 AM
C – Lecture	10:00 AM	11:00 AM
D – Client Call	11:00 AM	12:00 PM
E – Lunch Meeting	11:30 AM	12:30 PM
F – Workshop	12:00 PM	1:00 PM

Selected events using the Greedy strategy:

Event	Time	Decision
A – Team Meeting	9:00–10:00 AM	Selected
C – Lecture	10:00–11:00 AM	Selected
D – Client Call	11:00 AM–12:00 PM	Selected

F – Workshop	12:00–1:00 PM	Selected

The selected events do not overlap. Four events can be attended during the specified period.

6. Algorithm

Greedy Activity Selection Algorithm

1. Start with the list of calendar events.
2. Sort all events by their finishing times in ascending order.
3. Select the event that finishes earliest.
4. Examine the remaining events in sorted order.
5. Select an event if its start time is greater than or equal to the finish time of the last selected event.
6. Repeat until all events have been examined.
7. Display the selected events.
8. Stop.

7. Python Implementation

```
def select_events(events):
    # Each event is (name, start_time, end_time)
    events.sort(key=lambda event: event[2])

    selected = []
    last_end = -1

    for event in events:
        name, start, end = event

        if start >= last_end:
            selected.append(event)
            last_end = end

    return selected


events = [
    ("Team Meeting", 9, 10),
    ("Project Review", 9.5, 11),
    ("Lecture", 10, 11),
    ("Client Call", 11, 12),
    ("Lunch Meeting", 11.5, 12.5),
    ("Workshop", 12, 13)
]

selected_events = select_events(events)

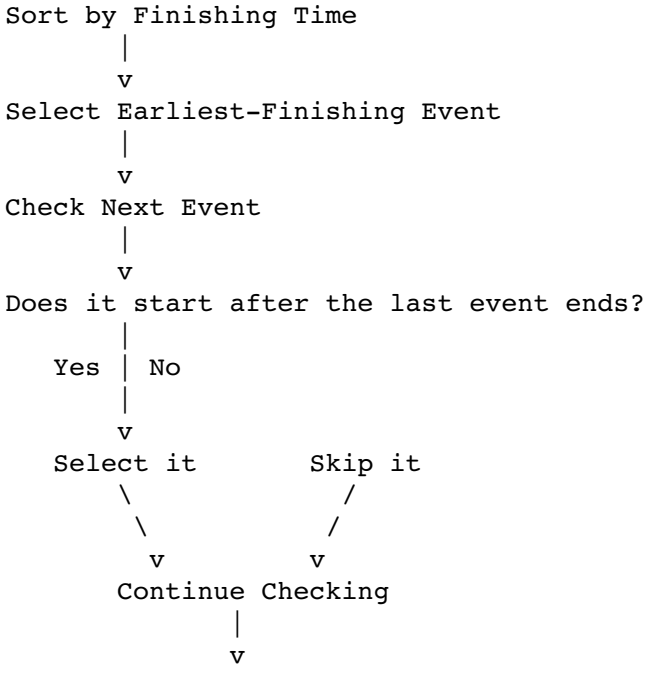
print("Selected Calendar Events:")
for name, start, end in selected_events:
    print(f"{name}: {start}:00 to {end}:00")
```

Note: The sample uses numeric hours for simplicity. A production implementation should use proper date-time values and handle dates, time zones, and event conflicts.

8. Working of the Algorithm

First, the events are sorted by finishing time. The algorithm selects the earliest-finishing event and then checks whether each remaining event starts at or after the end of the previously selected event.

```
Available Events
       |
       v
```



For the example, the algorithm selects:

- Team Meeting: 9:00–10:00 AM
- Lecture: 10:00–11:00 AM
- Client Call: 11:00 AM–12:00 PM